

K.E. Society's
RAJARAMBAPU INSTITUTE OF TECHNOLOGY , RAJARAMNAGAR
(An Empowered autonomous Institute, Affiliated to Shivaji University)

Department of Computer Science and Engineering

MINI PROJECT REPORT

SMART LAPTOP BUYING ASSISTANT

A Machine Learning Based Laptop Price Prediction,
Specification Evaluation and Usage Recommendation System

Submitted by

Akanksha Salunkhe - 2403064

Diksha Suryvanshi - 2553006

Manali Sawant - 2553009

Pranoti Bauchkar - 2553014

Under the guidance of

Prof. Snehal Patil

Academic Year: 2026–2027

INDEX

Sr. No.	Chapter / Section	Page No.
1	Introduction	3
2	Problem Statement	3
3	Objectives	3-4
4	Proposed System	4
5	Methodology	4-5
6	System Requirements	5
7	Dataset Description	6
8	AI Algorithm	6-7
9	Implementation	7-8
10	Results and Discussion	8-9
11	Advantages and Limitations	9
12	Applications	9-10
13	Future Scope	10
14	Conclusion	10
15	References	11

1. INTRODUCTION

Choosing a suitable laptop can be difficult because the market contains many brands, processor families, memory configurations, storage options, graphics units, display types, and operating systems. Prices vary considerably across configurations, and a high-priced laptop is not necessarily the best choice for every user.

The Smart Laptop Buying Assistant is a web-based application that uses machine learning to support laptop evaluation. A user selects laptop attributes through a web form, and the application estimates a market price, presents a specification rating on a five-point scale, and provides usage recommendations based on available hardware details. The application also presents technical specifications and similar laptop options to help users make an informed decision.

The system combines a Python-based machine learning workflow with a Flask web application. HTML and CSS provide the user interface, while pandas and the saved model pipeline support data handling and prediction. The aim is to make laptop comparison easier for students, developers, and general consumers.

2. PROBLEM STATEMENT

Laptop buyers often find it difficult to estimate a reasonable price and determine whether a configuration is suitable for their intended workload. Product specifications are presented in different formats, making direct comparison challenging. Buyers may purchase systems that are underpowered for programming, data science, gaming, or video editing, or spend more than necessary for their requirements.

The problem is to develop a simple web-based decision-support system that accepts selected laptop features, predicts an estimated price using a trained machine learning model, evaluates specifications, and communicates potential use cases in an understandable format.

3. OBJECTIVES

- Develop a user-friendly web application for laptop analysis.
- Estimate laptop price from selected configuration attributes using a trained machine learning model.

- Display a specification rating on a scale of 5, based on the project's rating model or available dataset rating.
- Provide understandable usage recommendations based on hardware attributes.
- Display detailed laptop specifications and similar laptop records.
- Integrate the trained model with a Flask application.
- Present results clearly so users can compare options before purchasing.

4. PROPOSED SYSTEM

The proposed system provides a web interface through which a user selects available brand, processor, RAM configuration, and operating system options. When the user submits the form, Flask validates the input and passes the expected features to the saved price-prediction pipeline. The predicted price is displayed on the analysis summary page.

The application uses available dataset records to show technical details and related laptop options. A specification-rating model or dataset rating supplies the rating shown to the user. Usage recommendations are generated from available specifications and configured suitability rules. Separate pages display the analysis summary, specifications, recommendations, and similar laptops.

- Input: selected laptop configuration attributes.
- Processing: validation, feature preparation, model inference, and matching relevant dataset records.
- Output: estimated price, specification rating, buying verdict, specifications, usage recommendations, and similar laptop records.

5. METHODOLOGY

The project follows a data-driven development process. The dataset is inspected and prepared, relevant input attributes are selected, and machine learning models are trained and evaluated. A trained pipeline is saved and loaded by the Flask application. The web interface collects user inputs and presents the generated results.

- Data collection: use the supplied laptop dataset containing brand, price, processor, memory, storage, graphics, display, operating system, and rating information where available.

- Data cleaning: inspect missing values, inconsistent categories, data types, and duplicate records.
- Feature preparation: select the features required by the trained model and apply the same preprocessing used during training.
- Model training: train and compare suitable regression algorithms for price prediction and specification rating.
- Model evaluation: use R^2, MAE, and RMSE on held-out data.
- Model persistence: save the fitted pipeline with joblib so it can be loaded by Flask.
- Web integration: create Flask routes and HTML templates to collect inputs, invoke prediction, and display results.
- Testing: verify form submission, predictions, page navigation, dropdown options, and template variables.

6. SYSTEM REQUIREMENTS

The application is intended to run on a standard personal computer with a modern web browser. Principal requirements are listed below.

Category	Requirement
Hardware	Laptop or desktop; 4 GB RAM minimum recommended, 8 GB or more preferred; sufficient storage for Python packages, dataset, and model files.
Operating System	Windows, Linux, or macOS.
Programming Language	Python 3.x.
Backend Framework	Flask.
Data Processing	pandas and NumPy.
Machine Learning	scikit-learn and any additional library required by the saved model, such as XGBoost.
Model Persistence	joblib or the serialization method used by the model.
Frontend	HTML, CSS, and JavaScript if used.
Development Tools	VS Code or another Python-compatible editor; terminal or command prompt.

7. DATASET DESCRIPTION

The project uses a laptop dataset supplied for model development and application integration. It contains information about laptop brands, product names, prices, hardware configurations, display properties, operating systems, warranty, and ratings. Exact availability may vary between raw and processed files.

The following table summarizes important attributes referenced by the application. Verify the final dataset headers before submission.

Attribute	Description
brand	Manufacturer or laptop brand.
name	Laptop product or model name.
price	Observed price; target variable for price prediction.
spec_rating / rating_5	Specification or user rating field, depending on the selected dataset/model.
processor / CPU	Processor family or CPU description.
Ram / Ram_type	Memory capacity and memory type.
ROM / ROM_type	Storage capacity and storage type.
GPU	Integrated or dedicated graphics information.
display_size	Screen size.
resolution_width / resolution_height	Horizontal and vertical display resolution.
OS	Operating system category.
warranty	Warranty information, when available.

The processed dataset should be used consistently with the preprocessing pipeline. State the final row count, exact feature count, and dataset source only after verifying the actual file used for training.

8. AI ALGORITHM

The project uses supervised machine learning for regression because laptop price and specification rating are numerical outputs. The exact algorithm for each output must match the model actually trained and saved in the project.

8.1 Price Prediction

Price prediction estimates a laptop's market price from selected configuration features. A regression pipeline may include preprocessing of categorical variables and a regression estimator. The trained pipeline is loaded at runtime and receives input in the same feature format used during training.

The reported evaluation for one price-model run was $R^2 = 0.8637$, MAE ≈ ₹12,817.77, and RMSE ≈ ₹21,613.12. Include these values only if they correspond to the final saved model and held-out evaluation set.

8.2 Specification Rating

A separate rating model or rating field is used to present a specification score out of 5. In development experiments, an XGBoost-based rating model was reported with $R^2 \approx 0.7969$. Confirm that this is the final model used by the application before reporting it as the deployed result.

R^2 indicates the proportion of target variation explained by the model relative to a baseline, while MAE and RMSE measure prediction error. These metrics should be computed on data not used to fit the model.

8.3 Usage Recommendations

Usage recommendations translate hardware characteristics into practical categories such as everyday computing, programming, web development, machine learning coursework, gaming, and video editing. The application may use rule-based thresholds for RAM, processor class, and graphics hardware. These recommendations should be described as specification-based heuristics, not as predictions from a separately trained recommendation model unless one has actually been implemented.

9. IMPLEMENTATION

The application is implemented as a Flask web application. The landing page provides dropdowns for supported laptop features. On form submission, the prediction route reads and validates the inputs, creates a data structure with the exact feature names expected by the saved pipeline, and invokes the model's predict method.

The application then selects a relevant laptop record from the dataset for presenting specification details and usage recommendations. The analysis summary presents the estimated price, rating, and verdict. Other Flask routes render the specifications, recommendations, and similar-laptops pages.

The user interface is implemented with HTML templates and CSS. Jinja inserts values from Flask into templates. The model is loaded with joblib when the application starts. The application can be run locally after installing the dependencies used by the project.

- Backend: Python and Flask.
- Data handling: pandas.
- Inference: saved machine learning pipeline loaded with joblib.
- Frontend: HTML, CSS, and Jinja templates.
- Pages: analysis summary, laptop specifications, usage recommendations, and similar laptops.

10. RESULTS AND DISCUSSION

The application provides an end-to-end interface for entering laptop configuration details and viewing analysis results. Its main outputs are an estimated price, specification rating out of 5, buying verdict, usage recommendations, detailed specifications, and similar laptop records.

In the reported price-model experiment, R^2 was approximately 0.8637, MAE was approximately ₹12,817.77, and RMSE was approximately ₹21,613.12. These results suggest that the model captured a substantial portion of price variation in the evaluation data, although absolute errors may still be meaningful for lower-priced laptops. RMSE being higher than MAE indicates that some predictions may have larger errors.

An XGBoost specification-rating experiment was reported with R^2 of approximately 0.7969. Present this value only after verifying the final evaluation notebook and deployed model. Results depend on the data split, preprocessing, feature selection, and exact trained model.

The system is a decision-support tool, not a guarantee of the current selling price or actual performance in every workload. Prices change over time, and real-world performance also depends on cooling, power limits, build quality, drivers, and workload characteristics.

Task	Reported metric	Reported value
Price prediction	R^2	0.8637
Price prediction	MAE	₹12,817.77
Price prediction	RMSE	₹21,613.12

Specification rating (XGBoost experiment)	R^2	0.7969; verify final deployed model

11. ADVANTAGES AND LIMITATIONS

11.1 Advantages

- Provides an accessible browser-based interface.
- Estimates laptop prices using a trained regression model.
- Presents technical information and a specification rating in a compact format.
- Helps users relate hardware configurations to potential workloads.
- Provides similar laptop records to support comparison.
- Separates the user interface from model inference through Flask routes and templates.

11.2 Limitations

- Predicted prices depend on the quality, coverage, and freshness of the training dataset.
- Prediction errors may be significant for unusual configurations or rapidly changing market prices.
- The specification rating depends on the definition and reliability of the rating target.
- Rule-based usage recommendations are approximate and cannot fully capture real-world performance.
- The current form uses a limited set of input features; additional attributes may improve coverage if the model is retrained appropriately.
- The application is a decision-support tool and should not replace current retailer listings or independent product reviews.

11. APPLICATIONS

- Students comparing laptops for programming, coursework, and project development.
- Developers evaluating configurations for software development and multitasking.
- Data science learners considering memory and processing requirements.
- General consumers estimating whether a listed price is reasonable.
- Laptop comparison portals or educational demonstrations of machine learning deployment.

- Academic demonstrations of regression models integrated with a web application.

12. FUTURE SCOPE

- Integrate periodically updated market prices from reliable, permitted data sources.
- Expand the dataset to cover more brands, product generations, and configurations.
- Evaluate additional regression algorithms and use consistent cross-validation.
- Improve the specification-rating model with a clearly defined, validated target.
- Replace basic suitability rules with a transparent workload-specific scoring approach supported by benchmark data.
- Add filtering by budget and user needs while retaining the existing prediction workflow.
- Deploy the application to a secure cloud platform and add input validation, logging, and model-version tracking.
- Provide model explainability, such as feature importance, to help users understand which attributes influence predicted price.

13. CONCLUSION

The Smart Laptop Buying Assistant demonstrates how machine learning can be integrated into a web application to support laptop purchasing decisions. The system combines price prediction, specification evaluation, hardware-based usage recommendations, and laptop comparison in a single interface.

The project uses Python for data processing and model inference, Flask for web routing, and HTML/CSS for the user interface. Reported experimental results indicate that the price model can explain a substantial amount of price variation, while the rating model's performance should be confirmed against the final deployed artifact. The system remains limited by dataset coverage, price changes, and the approximate nature of workload recommendations.

Overall, the project provides a practical foundation for a laptop decision-support application and can be improved further through larger datasets, rigorous evaluation, transparent recommendations, and deployment testing.

14. REFERENCES

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- pandas documentation. https://pandas.pydata.org/docs/
- Flask documentation. https://flask.palletsprojects.com/
- Joblib documentation. https://joblib.readthedocs.io/
- Scikit-learn documentation: Model evaluation and regression metrics. https://scikit-learn.org/stable/modules/model_evaluation.html
- Project dataset: add the original source URL, creator, and access date for the laptop dataset used in model development.